FixProof | Four-Member Work Division

Team execution plan • Based on the merged live-investigation milestone (29 Oct 2026) • Anonymous allocation - assign Member 1-4 internally

Current baseline

The read-only flow is locally demonstrated: authenticated incident intake → durable worker → scoped MCP evidence → persisted observations → structured Nemotron analysis → operator UI. The remaining critical path is exact proposal/approval, guarded Kubernetes restore, independent recovery verification, and repeatable release/demo acceptance. Local tests are documented; full mutation/recovery acceptance is not yet complete.

Ownership at a glance

Member	Primary workstream	Key deliverable
1 - Backend & approval	Phase 5 API, data models, trust boundaries	Immutable, evidence-backed proposal and authenticated approval/rejection
2 - Kubernetes & execution	Phase 6 MCP/RBAC, preflight, rollback	Single scoped restore action with replay and drift protection
3 - AI & investigation quality	Structured findings, grounding, ChatOps quality	Reliable evidence-to-proposal handoff; no unsupported claims
4 - Frontend, verification & QA	Phase 7 UI, verification orchestrator, end-to-end testing	Measured recovery verdict, operator timeline, demo-ready tests

Non-negotiable shared rules

- Keep one registered demo Deployment as the only mutation target; do not enable arbitrary clusters/commands.
- Model output is advisory: neither Nemotron nor ChatOps can select a trusted baseline, approve, or execute.
- No Kubernetes write path may ship until exact approval, preconditions, replay protection and negative tests pass.
- Every PR must contain tests, updated typed API contracts where relevant, and a concise handoff with observed results.
- Do not interpret a successful patch as verified recovery; only new post-action observations may prove it.

Individual responsibilities

MEMBER 1 | Backend: trusted proposal & approval

Own the authoritative FastAPI lifecycle and PostgreSQL migrations, not Kubernetes credentials.

- Persist a trusted known-good template/hash independently of the AI output; bind it to exact cluster, namespace, Deployment UID.
- Build immutable proposal versions containing evidence IDs, expected faulty template hash, Kubernetes resourceVersion, desired good template hash, expiry and canonical SHA-256 hash.
- Create authenticated review/approve/reject endpoints with CSRF, idempotency and audit; invalidate on target/payload drift or expiry.
- Define typed request/response models, migrations and generated frontend contracts; include concurrency and replay tests.

Exit gate: An operator can approve or reject only the exact current proposal. Changed, expired, rejected, unknown-citation or wrong-target proposals are non-executable; no infrastructure mutation.

MEMBER 2 | Infrastructure: one guarded restore action

Own MCP Kubernetes integration, scoped execution capability and Kubernetes safety.

- Design a namespace- and resource-scoped write identity for sample-api; retain separate read/write privileges.
- Implement one-use, short-lived API-issued execution capability bound to approved proposal, target and execution ID; independently validate in MCP.
- Check UID, resourceVersion, expected bad-template hash and trusted good-template hash immediately before atomic Kubernetes patch.
- Store action result and reconcile unknown/pending outcomes instead of blindly retrying; test stale target, denied scope and replay.

Exit gate: Exactly one valid approved action patches only the owned sample Deployment. Unauthorized, stale, replayed or changed actions cause zero mutation.

MEMBER 3 | AI: evidence quality & recommendation boundary

Own the model contracts and investigation quality, but never approvals or Kubernetes writes.

- Review structured Nemotron responses, supported/contradicting evidence UUIDs, missing-source handling and calibrated uncertainty.
- Bridge validated advisory action choices to Member 1 proposal creation without allowing AI-generated YAML, targets or baseline selection.
- Complete or explicitly defer grounded ChatOps: pass retained evidence, persist dynamic tool reads and validate all citations.
- Test malformed outputs, prompt injection in logs, model unavailability, cost/time budgets and evidence/source drift; clarify legacy graph vs integrated worker strategy.

Exit gate: Every model finding references retained evidence; invalid outputs produce needs_review and never create an approvable live proposal.

MEMBER 4 | Frontend + verification + release QA

Own the operator experience and independent recovery decision; coordinate shared verification API contracts with Member 1.

- Build a clear proposal comparison and deliberate approve/reject UI; never imply AI recommendation equals authorization.
- Implement independently measured post-action verification: rollout available, 3 fresh /healthz checks, 60-second window with >=20 work requests and <1% 5xx.
- Persist verdict and actual measured counts, time windows, probes, revision and reason: passed / failed / inconclusive.
- Complete timeline from trigger to verification; add browser tests, failure states and clean-machine demo rehearsals.

Exit gate: UI only shows verified recovery after passing fresh measurements; missing/stale metrics are inconclusive. E2E positive and negative tests are reproducible.

Integration sequence & handoffs

Work in parallel, integrate in this order

Gate	Owners	Required handoff / acceptance
A. Contract freeze	1 + 2 + 3 + 4	Agree proposal JSON schema, evidence IDs, trusted baseline source, allowed action enum, verification states, API paths.
B. Read-only proposal	1 + 3; 4 UI	AI outputs validated suggestion; API independently constructs immutable exact proposal; UI previews without mutation.
C. Human approval	1 + 4	Operator reviews evidence + exact change, approves/rejects; auth, hash, expiry and drift denial tests pass.
D. Controlled patch	2 + 1	MCP enables one write tool only after capability, preflight and reconciliation tests; execute one approved patch.
E. Measured verdict	4 + 2	New window starts after patch; verify rollout + health probes + traffic; persist passed/failed/inconclusive.
F. Release demo	All	Run full scenario twice, plus rejection, expired approval, target drift, provider outage and insufficient-traffic cases.

Suggested PR boundaries (avoid one giant PR)

- PR A - Proposal contracts + trusted baseline + read-only API (Member 1 with Member 3 review).
- PR B - Approval API + proposal-review UI + negative cases (Members 1 and 4).
- PR C - Scoped mutation capability + MCP preflight + reconciliation (Member 2, security review by Member 1).
- PR D - Post-action verification + full timeline + browser E2E (Member 4, supported by Member 2).
- PR E - Integration stabilization, CI, runbook and demo evidence (all four).

Demo story for the hackathon

- Show a healthy sample app on local kind, with live Prometheus telemetry.
- Inject a controlled configuration fault; show reproducible 5xx and failing functional health.
- Report from FixProof; show durable evidence and the Nemotron-supported analysis.
- Show the exact trusted restore proposal and deliberate operator approval.
- Run the restricted restore only if its safety gates are implemented and tested.
- Show fresh rollout/probe/error-rate readings and a verified recovery verdict; otherwise present remediation as future work, not completed.

Cloud Kubernetes / Civo decision

Optional, not a prerequisite for the main demo. Prefer the existing validated local kind fixture for the first complete incident-to-recovery run. A remote Civo (or other) cluster requires a separate target registration, connectivity, RBAC, identity and safety test plan. Assign remote-cluster proof of concept to Member 2 only after local mutation and verification acceptance.

Final completion checklist

Minimum product-complete acceptance

- [ ] All four owners have approved and tested their documented API/permission boundaries.
- [ ] Immutable proposal binds canonical evidence, exact Deployment UID, observed faulty state, trusted good state, hash and expiry.
- [ ] Approval and rejection are authenticated, audited and invalidate stale or changed proposals.
- [ ] MCP can patch only the registered sample Deployment after independently validating a one-use capability.
- [ ] A failed or interrupted execution is reconciled; no automatic replay of unknown outcomes.
- [ ] Fresh measurements yield passed / failed / inconclusive, and incident closure remains explicit.
- [ ] Full positive flow passes twice on a clean owned kind fixture; denial and failure cases pass.
- [ ] API SQLite/PostgreSQL, AI, MCP, web, contract and browser tests are green in CI.
- [ ] No model keys, operator secrets, kubeconfigs or raw sensitive logs leak into repo or demo records.
- [ ] Demo screen recording and speaker flow match actual features; no simulated UI labeled as live.

Known follow-ons - do not block the first demo

- Optional Prometheus auto-detection and broader incident types.
- Production multi-user login, cloud-hosted SaaS deployment and remote Civo integration.
- Advanced ChatOps, dynamic iterative retrieval and expanded diagnostic tools.
- Real cost/billing ingestion and broader Kubernetes mutation support.

How to use this plan

Assign one primary owner for each workstream and one reviewer from a different workstream. Have each member open a dedicated feature branch, land contract changes first, and integrate at Gates A-F. Prioritize demonstrable correctness over extra features.

Source snapshot: merged main d84dc06 (29 Oct 2026); NEXT.md and LIVE_INVESTIGATION_HANDOFF.md. Roles are placeholders only. No personal identities or usernames included.